

04. 기술 비교 및 논리 아키텍처

「돌아가는 판단」1차 프로젝트

On-premise · Kubernetes · Ansible 기반 기술 선택과 논리 구조 기준서

목차

1. 문서 목적과 판단 범위
2. 설계 입력과 추적 기준
3. 기술 선택 평가 기준과 재사용 원칙
4. 최종 논리 역할·아키텍처 영역 기술 매핑
5. Kubernetes Platform과 Common VIP 기반 서비스 진입
6. Application Runtime과 AI 판세 분석 실행 경계
7. Runtime State·Realtime 처리
8. Persistent Data: MariaDB Primary/Replica + MaxScale
9. Persistent Storage와 Storage 후보 검토
10. CI/CD·GitOps 및 Image Delivery
11. Infrastructure Automation
12. Observability
13. Configuration·Security
14. DR·Backup·Recovery
15. 장애 영향·동시성·AI 격리 검증
16. 전체 논리 아키텍처
17. 최종 선택 요약 및 후속 설계 이관

1. 문서 목적과 판단 범위

서비스 요구사항 및 기능 명세, 핵심 문제 및 검증 목표, 논리 역할·서비스 목록에서 확정된 요구와 책임을 실제 구현 기술 및 논리 구조로 연결한다. 기술은 선행 가정이 아니라 정확성·복구성·검증 가능성·구현 가능성을 만족하기 위한 수단으로 선택한다.

이 문서에서 확정	후속 물리·구현 설계에서 확정
Runtime Platform, Gateway, CNI, Application, State, Data, Storage, CI/CD, Observability, Automation의 기술 선택	Physical Server/VM 배치, CPU·Memory·Disk, Worker 수용량
Common VIP 10.1.93.90 + Keepalived VRRP + HAProxy Port별 L4 진입 구조	LB-01/LB-02의 실제 배치·Failure Domain, NIC, Firewall Rule
MariaDB Primary/Replica + MaxScale 2대의 원 목표 논리 구조	DB/MaxScale의 실제 서버 배치와 장애 전환 절차 세부
Jenkins·Argo CD의 Kubernetes 내부 실행, Harbor 외부 배치라는 책임 경계	Pod/Replica/PVC/Resource 및 실제 VM 배치
ANALYSIS의 게임 권위 경로 분리와 실행 경계	분석 알고리즘·모델 세부 및 Resource Request/Limit
Backup/Restore의 대상과 도구 방향, 장애 검증 지점	Backup 주기·Retention·실험 부하·판정 임계값

원 목표 구조 이 문서는 1차 프로젝트의 원 목표 구조를 정의한다. 이후 최소 구현 범위를 도출하더라도 여기의 구조를 임의로 축소해 섞지 않는다.

2. 설계 입력과 추적 기준

서비스 측에서는 Room·Game·Turn·Vote·Move·Result의 서버 권위 규칙과 Member/Guest 저장 경계, 30초 재접속, Pass, 렌즈 금수, AI 판세 분석 계약이 고정되어 있다. 검증 측에서는 상태 일관성, 동시성 정확성, Backend 장애 연속성, 집중 부하, On-premise 자동화, 영속 데이터 DR이 핵심 문제로 정의되어 있다. 논리 역할 측에서는 WEB, IDENTITY, ROOM, GAME, VOTE, ANALYSIS, REALTIME, STATE, DATA, MONITORING, LOGGING, AUTOMATION과 횡단 HEALTH 책임이 분리되어 있다.

핵심 문제	논리 책임	구조적 대응	검증 관점
실시간 상태 일관성	STATE + ROOM/GAME/VOTE/REALTIME	Redis Shared Runtime State + Backend Replica	어느 Backend에서도 동일 Room/Game/Turn/Board 상태 복원
동시 Vote·단일 Move	VOTE → GAME → DATA	Redis Vote State + Application 역등성 + MariaDB Transaction/Constraint	정상 Turn Move 1개, Pass Turn 0개, stale 상태 변경 0건
Backend 장애 연속성	REALTIME + STATE + HEALTH	Kubernetes Replica + Service + Redis State + 재연결	장애가 사용자 패배로 오픈되지 않고 상태 복원
집중 부하·AI 격리	VOTE + REALTIME + ANALYSIS + MONITORING	Scale-out 가능한 Backend + Redis Streams 기반 독립 ANALYSIS 실행 경계 + Metrics	Vote 경로 지연과 AI 처리 영향을 함께 측정
On-premise 반복 구축	AUTOMATION	Ansible + kubeadm + 외부 인프라 자동화	수동 대비 구축·재구축 시간·개입 비교
영속 데이터 손실·DR	DATA + AUTOMATION +	MariaDB Backup + Redis AOF +	Restore·무결성·RTO/RPO 측정

핵심 문제	논리 책임	구조적 대응	검증 관점
	HEALTH	etcd Snapshot	

3. 기술 선택 평가 기준과 재사용 원칙

기준	판단 질문	우선순위
정확성·일관성	동시 요청에도 Vote·Move·Result가 중복·유실되지 않는가	최우선
장애 복구성	단일 구성요소 장애 후 상태와 서비스를 복원할 수 있는가	최우선
상태·데이터 경계	Runtime State와 영속 기록의 소유·수명·복구 범위가 명확한가	최우선
구현 가능성	4인 팀과 1차 프로젝트 기간, On-premise에서 구축·시험·설명이 가능한가	최우선
성능·확장성	집중 이벤트·동시 연결·AI 분석 부하의 병목을 측정하고 확장 가능한가	중요
자동화·재현성	Ansible로 반복 구축·설정·복구할 수 있는가	중요
관측·검증성	핵심 주장을 Metrics/Logs/시간표식으로 증명할 수 있는가	중요
복잡도·확장 균형	1차 성공을 해치지 않고 향후 확장을 불필요하게 막지 않는가	통제 기준

3.1 공식·OSS 자산 사용 원칙

공식 Container Image, Manifest, Helm Chart, Installer, 검증된 Exporter와 Dashboard 등은 프로젝트 핵심 역량을 훼손하지 않는 범위에서 재사용할 수 있다. 직접 구현 여부는 “직접 만들 수 있는가”가 아니라 “직접 만드는 과정이 아키텍처·자동화·통합·장애·정량 검증이라는 핵심 가치를 증명하는가”로 판단한다.

- 아키텍처 책임 경계, Ansible 자동화, 장애 시나리오와 정량 검증은 직접 설계한다.
- 공식 설치 자산을 활용하더라도 적용 순서, 변수, 보안, Health 검증, 재실행 정책은 프로젝트 코드와 문서로 관리한다.
- 기존 Dashboard를 활용하는 경우 핵심 KPI가 빠지면 프로젝트용 Dashboard로 보강한다.

4. 최종 논리 역할·아키텍처 영역 기술 매핑

논리 역할·아키텍처 영역	선택 기술·구조	책임 경계
WEB / External Access	Gateway API + NGINX Gateway Fabric / Frontend Nginx	HTTP/HTTPS/WebSocket 진입, L7 Route, 정적 UI 제공
IDENTITY / ROOM / GAME / VOTE / REALTIME	Python + FastAPI Modular Monolith	인증·Room·Game·Vote·Realtime API의 권위 비즈니스 판단
ANALYSIS	Kubernetes 내부 독립 ANALYSIS Workload + Redis Streams Consumer Group	공식 Move 이후 비동기 Job 처리. 결과는 game_id + move_no에 귀속되며 재처리는 멍등하게 수렴하고 Game 권위

논리 역할·아키텍처 영역	선택 기술·구조	책임 경계
		판단에 영향 없음
STATE	Redis + AOF + Pub/Sub	Room/Game/Turn/Vote/GuestSession/Board Analysis 최신 상태와 IDENTITY가 관리하는 Member 인증 세션을 공유·복원하고 Realtime·복구 기반 제공
DATA	MariaDB Primary/Replica + MaxScale 2	영속 데이터 무결성, DB 접근·Routing, 복제 기반 장애 전환 경계
PLATFORM	Kubernetes + kubeadm / Calico / CoreDNS	Workload 실행, Service, Network, Cluster 제어
PERSISTENT STORAGE	External NFS + NFS Subdir External Provisioner + StorageClass/PVC/PV	기존 NFS Share를 사용해 PVC별 하위 디렉터리/PV를 동적 프로비저닝하는 Shared File Storage. Workload Local Storage와 책임을 분리
CI/CD	Jenkins(in-cluster) + Harbor(external) + Argo CD(in-cluster)	Build/Test, Image Registry, Git Desired State 배포 책임 분리
MONITORING	Prometheus + Alertmanager + Grafana + Exporters	핵심 KPI·진단 Metric 수집, Alert Rule 평가, 운영자 대응이 필요한 상태의 알람 라우팅, 조회·시각화
LOGGING	Grafana Alloy + Loki	Application/Kubernetes/System 로그 수집·중앙 조회
AUTOMATION	Ansible	Host·Cluster·External Infra 구축·설정·복구 보조
HEALTH	Application Health Endpoint + Kubernetes Probe + Component Health Check	Backend·ANALYSIS·Platform·External Service가 각자 Health 상태를 제공하고 Kubernetes, HAProxy, Monitoring, Automation이 이를 장애 감지·복구·검증에 사용. 독립 비즈니스 서비스로 배포하지 않는 횡단 운영 책임

역할 ≠ 배포 단위 FastAPI 내부의 IDENTITY/ROOM/GAME/VOTE/REALTIME은 코드 책임을 분리하되 하나의 Backend 배포 단위로 유지한다. ANALYSIS는 실패·자원 격리 필요 때문에 별도 실행 경계를 선택하지만, 물리 서버 분리를 의미하지 않는다.

5. Kubernetes Platform과 Common VIP 기반 서비스 진입

5.1 Runtime Platform 결정

후보	장점	제약	판정
VM 직접 배포	구성이 단순하고 Runtime 계층이 적음	Replica·Service Discovery·재스케줄·GitOps·장애 실험을 별도 구성	제외
Kubernetes(kubeadm)	Replica, Service, Health, 표준 Network/Storage, Node 장애 재스케줄과 Scale-out 실험 가능	Control Plane/CNI/Storage 등 구축 복잡도 증가	선택

Kubernetes는 서비스 기능 자체를 위한 장식이 아니라 Backend 장애 연속성과 Scale-out을 On-premise에서 반복 검증하기 위한 Runtime Platform으로 사용한다. 관리형 Cloud 기능에 의존하지 않고 kubeadm 기반으로 구축해 Ansible 자동화 가치와 연결한다.

5.2 Cluster Network 결정

후보	판정	이유
Flannel	제외	Pod 연결은 단순하지만 NetworkPolicy 검증 범위가 약함
Calico	선택	CNI와 NetworkPolicy를 함께 제공해 Application·Platform·Observability 간 필요한 통신 경계를 표현 가능
Cilium	제외	고급 네트워크·관측 기능은 장점이나 현재 요구 대비 기능 중복과 학습 범위가 큼

Service Discovery는 CoreDNS를 사용한다. Namespace는 책임 구분을 돕지만 보안 경계로 간주하지 않으며 NetworkPolicy와 RBAC로 실제 접근 경계를 보완한다.

5.3 외부 진입과 Common VIP

Endpoint	논리 목적	내부 전달
10.1.93.90:80	HTTP 진입 및 HTTPS Redirect	HAProxy L4 → Kubernetes Gateway API / NGINX Gateway Fabric → HTTPS Redirect
10.1.93.90:443	HTTPS / WebSocket 서비스 진입	HAProxy L4 → Kubernetes Gateway API / NGINX Gateway Fabric → Frontend·Backend Service
10.1.93.90:6443	Kubernetes API 진입	HAProxy L4 계층 → Kubernetes Control Plane API Endpoint
10.1.93.90:3306	DB Access Endpoint	HAProxy L4 계층 → MaxScale 2대 → MariaDB Primary/Replica

Common VIP 10.1.93.90은 Keepalived의 VRRP로 LB 계층 사이에서 소유권을 전환하고, HAProxy가 :80/:443, :6443, :3306의 L4 전달과 Backend Health Check를 담당한다. Keepalived는 VIP 가용성, HAProxy는 L4 Proxy와 Health Check에 집중하여 책임 중복을 피한다.

L4/VIP 후보	판정	이유
HAProxy + Keepalived	선택	Keepalived VRRP로 Common VIP를 전환하고 HAProxy TCP mode와 Health Check로 포트별 Backend를 전달한다. VIP와 L4 Proxy 책임을 분리해 장애 검증이 명확하다.
HAProxy 단독	제외	L4 Proxy와 Health Check는 가능하지만 Common VIP의 노드 간 소유권 전환을 별도로 해결해야 한다.

Keepalived/IPVS 중심	제외	VIP와 Load Balancing을 한 계층에 집중할 수 있으나 HAProxy와 역할이 중복되고 현재 구조의 Health/Routing 설명이 복잡해진다.
--------------------	----	---

Common VIP 하나를 포트로 분리하여 서비스·Kubernetes API·DB Endpoint를 구분한다. Keepalived는 LB 계층에서 10.1.93.90의 VRRP Failover를 담당하고 HAProxy는 포트별 L4 전달과 Backend Health Check를 수행한다. 이는 세 개의 별도 VIP를 사용하는 구조가 아니다. :3306은 서비스 사용자에게 공개하는 포트가 아니라 필요한 Backend 및 관리 경로에만 허용하는 DB 접근 경계이며, 구체적인 Source/Destination·Firewall 규칙과 LB-01/LB-02의 물리 배치는 물리 아키텍처에서 확정한다.

Common VIP와 Keepalived VRRP는 원 목표 구조로 선택한다. 실제 적용 전 물리 아키텍처 단계에서 LB 간 L2 연결, VRRP 통신, VIP 이동 및 Gratuitous ARP 동작 가능 여부를 확인한다. 교육장 네트워크 제약으로 사용할 수 없는 경우 MVP에서는 단일 LB 고정 IP 구조를 적용하고, VIP 자동 전환 검증은 제외 범위로 기록한다.

L7 후보	판정	이유
별도 Nginx/HAProxy Reverse Proxy	제외	NGINX Gateway Fabric과 HTTP/WebSocket L7 Routing 책임 중복
Ingress 기반	대안	성숙하지만 Gateway/Route 역할 분리와 정책 확장성에서 Gateway API보다 우선순위가 낮음
Gateway API + NGINX Gateway Fabric	선택	Gateway와 Route 책임을 분리하고 HTTP/WebSocket 진입을 Kubernetes 표준 구조로 일원화

서비스 진입 논리 흐름

USER → DNS → Common VIP 10.1.93.90:443 → Keepalived VRRP가 활성 LB의 VIP 소유권 유지 → HAProxy L4 → Gateway API / NGINX Gateway Fabric → Frontend Service 또는 Backend Service → Application Roles / REALTIME

※ :80은 HTTP 요청을 HTTPS로 전환하기 위한 진입점으로 사용한다.

6. Application Runtime과 AI 판세 분석 실행 경계

6.1 Backend 구현 단위

후보	판정	이유
역할별 Microservice	제외	논리 역할 수만큼 배포 단위를 분리하면 네트워크·배포·장애 경계가 늘어나 핵심 검증보다 운영 복잡도가 커짐
FastAPI Modular Monolith	선택	IDENTITY/ROOM/GAME/VOTE/REALTIME 책임은 코드 모듈로 분리하면서 배포 단위는 단순하게 유지
단일 무구조 Backend	제외	책임 경계·동시성·복구 시험의 추적성이 약해짐

Frontend는 Nginx 기반 Deployment + Service, Backend는 FastAPI Deployment + Service로 구성한다. Backend는 Replica 확장이 가능해야 하며 권위 상태가 특정 Pod의 로컬 메모리에 종속되지 않도록 한다.

IDENTITY 인증·세션 경계 Guest와 Member의 인증 상태가 특정 Backend Pod의 Local Memory에 종속되지 않도록 한다.

Guest와 Member의 활성 인증 세션은 Backend Replica가 공통으로 조회할 수 있는 Shared Session 구조를 사용하고, Member 계정과 password hash는 영속 DATA에 저장한다.

Session 구현 후보로 Backend Local Session, Stateless Token, Redis 기반 Server-side Session을 검토한다. Backend Local Session은 구현은 단순하지만 Replica 간 세션 공유와 Pod 장애 후 연속성에 불리하므로 제외한다. Stateless Token은 Backend 간 공유 저장소 의존을 줄일 수 있으나 Guest 임시 세션 만료, 명시적 로그아웃·세션 폐기, 활성 서비스 상태와의 수명 관리가 별도 문제로 남는다. Redis 기반 Server-side Session은 이미 Runtime State를 위해 사용하는 Redis를 재사용할 수 있고 Guest 임시 세션과 Member 인증 세션을 Replica 간 공유할 수 있어 현재 구조에 적합하므로 기본 방식으로 선택한다.

Client는 추측하기 어려운 Session Identifier만 전달하고, 실제 인증·사용자 상태는 서버가 확인한다. Member 비밀번호는 MariaDB의 영속 계정 데이터에 hash 형태로 저장하며 Session Store에 비밀번호 원문을 저장하지 않는다. 구체적인 Cookie 속성, Session TTL, password hashing 알고리즘과 Key 이름은 구현 단계에서 확정한다.

6.2 ANALYSIS 실행 경계 비교

후보	장점	문제	판정
Backend 동기 모듈	구현이 가장 단순	분석 지연·실패가 Move 이후 경로를 직접 지연시켜 게임 권위 처리와 결합	제외
Backend 내부 비동기 Task	호출 경로가 단순하고 별도 Service 불필요	Backend CPU/Memory·재시작 Failure Domain을 공유해 부하 격리가 제한	대안
Kubernetes 내부 독립 ANALYSIS Workload	게임 권위 경로와 실패·자원 경계를 분리하고 별도 Scale/Restart 가능	내부 통신과 배포 대상이 하나 증가	선택

ANALYSIS는 공식 MOVE_APPLIED 직후 확정 Board Snapshot을 대상으로 BLACK/WHITE 예상 승률을 산출한다. GAME은 분석 작업을 Redis Streams에 기록하고 해당 game_id + move_no의 analysis_status를 PENDING으로 갱신하며, 게임 판정과 다음 Turn은 분석 완료를 기다리지 않고 계속 진행한다. ANALYSIS Consumer Group은 작업을 비동기로 처리하고 성공 시 black_win_probability와 white_win_probability를 산출하여 analysis_status를 READY로 갱신한다. 두 승률 값은 합계가 100%가 되도록 정규화한다. 분석 처리 중 재시도 가능한 일시 실패가 발생하면 analysis_status를 PENDING으로 유지하고, 재시도 정책으로 회복하지 못한 최종 실패에만 FAILED로 갱신한다. 요청과 결과는 game_id + move_no로 식별하며 Pass에는 새 분석 작업을 생성하지 않는다. Redis Streams의 at-least-once 재처리 특성을 고려하여 동일 game_id + move_no 결과는 멍등하게 반영하고, 현재 표시 대상보다 과거 move_no의 늦은 결과는 최신 분석을 덮어쓰지 않는다. 분석 실패·지연은 Turn, Move, 렌주 판정, GameResult, Rating의 게임 권위 경로에 영향을 주지 않는다.

알고리즘·모델 경계 판세 분석 방식은 규칙·휴리스틱 기반 평가, 탐색 기반 평가, 학습 모델 기반 평가를 비교 대상으로 둔다. 1차 프로젝트에서는 실행시간의 예측 가능성, 구현 난이도, 재현성, 자원 요구와 프로젝트 중심 범위를 고려하여 규칙·휴리스틱 기반 평가를 기본 방식으로 선택한다. 탐색 기반 평가는 계산량 증가와 응답시간 변동 가능성이 있고, 학습 모델 기반 평가는 학습 데이터·모델 검증 범위가 추가되므로 현재 기본 구조에서는 채택하지 않는다. 구체 평가식·가중치·승률 변환 방식과 정확도 평가는 구현·검증 단계에서 확정한다.

6.3 ANALYSIS 작업 전달 방식

후보	장점	문제	판정
----	----	----	----

내부 HTTP 직접 호출	구현이 단순하고 호출 관계가 명확	ANALYSIS 일시 장애 시 호출자 Retry/Backoff와 작업 보존을 Backend가 직접 책임져야 함	대안
Redis Pub/Sub	기존 Redis 재사용과 전달 지연이 작음	구독자가 순간적으로 없으면 작업 전달을 보존하지 못해 Analysis Job 전달 용도에 부적합	제외
Redis Streams + Consumer Group	기존 Redis를 재사용하면서 Job 보존, Pending, ACK, 재처리 경계를 제공	at-least-once 특성 때문에 game_id + move_no 기준 멍등 결과 처리가 필요	선택
RabbitMQ/Kafka	전용 Queue/Streaming 기능과 확장성	AI 보조 기능 하나를 위해 별도 운영 Stack을 추가해 현재 범위 대비 복잡도가 큼	제외

Move 이후 분석 흐름

GAME: Move Commit + Board Snapshot 확정

→ 게임 판정 / 다음 Turn은 즉시 계속

↳ Redis Streams: analysis job 기록

(game_id + move_no + 해당 Move의 immutable Board Snapshot 또는 해당 Snapshot을 식별하는 불변 참조)

→ STATE: analysis_status=PENDING

→ ANALYSIS Consumer Group: 처리

↳ 성공: black/white 승률 산출 → analysis_status=READY → ACK

↳ 처리 실패·재시도 가능: analysis_status=PENDING 유지 → Pending·Retry

↳ 최종 실패: analysis_status=FAILED → 해당 분석 작업 종료

→ 현재 표시 대상 move_no와 일치하는 결과만 STATE의 최신 BoardAnalysis에 멍등 반영

→ Redis Pub/Sub → REALTIME → 사용자 표시

※ 같은 game_id + move_no 재처리는 멍등하게 처리하고 stale 결과는 현재 분석을 덮어쓰지 않음

7. Runtime State·Realtime 처리

7.1 Runtime State 저장소

후보	장점	문제	판정
Backend Local Memory	가장 단순	Replica 간 불일치, Pod 장애 시 Room/Game 상태 손실	제외
Persistent DB 중심	영속성과 단일 저장점	Vote/Turn 등 고빈도 Runtime State가 영속 DB에 집중되어 책임과 부하가 혼잡	제외
Redis Shared State	빠른 공유 상태 접근, TTL/자료구조, Pub/Sub 및 Streams 연계	Redis 장애 영향이 커지므로 AOF·복구 절차와 Stream Pending/재처리 검증 필요	선택

Redis는 Room, Participant, Ready, Board, Turn, Current Vote, GuestSession, BoardAnalysis의 현재 상태와 IDENTITY가 관리하는 Member 인증 세션을 Backend Replica 간 공유한다. Redis는 비즈니스 규칙을 결정하지 않으며 ROOM/GAME/VOTE/ANALYSIS가 확정한 상태를 여러 Backend가 동일하게 읽고 복원할 수 있도록 한다.

Redis HA 범위 Redis Sentinel/Cluster는 1차 원 목표 논리 구조의 기본 구성에 포함하지 않는다. 핵심은 Backend 다중화와 공유 상태 복원이며, Redis 자체 장애는 AOF와 복구 실험으로 영향·MTTR을 명시적으로 검증한다.

7.1.1 Vote 동시성 처리

후보	장점	문제	판정
Backend Local Lock	구현 단순	Replica별 Lock이 분리되어 다중 Backend 동시성 보장 불가	제외
별도 Distributed Lock	여러 Backend 조정 가능	Lock 획득·만료·복구 경계가 추가되어 현재 Vote 처리 대비 복잡도 증가	대안
Redis 원자 처리	현재 Vote State와 동일 저장소에서 Turn 상태 확인·투표 변경·마감 상태 전이를 하나의 원자 작업으로 처리 가능	원자 처리 Script/Transaction의 멍등성과 실패 시 재실행 검증 필요	선택

Vote 접수·변경·삭제와 deadline 전환은 Redis의 원자 처리 기능을 이용해 동일 game_id + turn_no의 현재 Turn 상태와 사용자별 마지막 유효표를 함께 검증·변경한다. Turn이 VOTING에서 RESOLVING으로 전환된 이후 동일 turn_no의 신규·변경 요청은 상태를 수정하지 않는다. 구체적인 Redis Script/Transaction 구현 방식과 Key 구조는 구현·검증 단계에서 확정한다. 최종 Move의 중복 방지는 별도로 Application 멍등 처리와 MariaDB Transaction/Constraint가 담당한다.

7.2 Realtime 전달

Client와 Backend 간 실시간 통신은 WebSocket을 사용한다. Backend Replica 간 상태 변경 알림은 Redis Pub/Sub을 사용하되 Pub/Sub 이벤트 자체를 최신 상태의 원본으로 간주하지 않는다. ANALYSIS 작업 전달은 보존과 ACK가 필요한 Redis Streams로 분리한다. 재접속이나 Realtime 이벤트 누락 시 Redis Current State를 다시 조회하여 최종 상태로 수렴한다.

- 연결 단절 시 REALTIME이 사건을 도메인에 전달하고 VOTE는 마감 전 현재 표를 제거한다.
- 30초 재접속 유예는 Turn을 정지시키지 않으며, 참가 상태 복원과 팀 전원 이탈 판정은 ROOM/GAME의 권위 상태에 따른다.
- 같은 게임의 사용자들은 Move·Turn·Board·AI 최신 결과를 동일한 원본 상태에서 재동기화한다.

8. Persistent Data: MariaDB Primary/Replica + MaxScale

8.1 데이터 계층 요구

Member, MemberStats, Move, GameResult, RatingHistory는 서비스 재시작 이후에도 보존되어야 한다. 동일 game_id + turn_no의 Move는 최대 하나이며 정상 Turn은 1개, Pass는 0개다. 동일 game_id의 GameResult·Member 전적·Rating 반영은 1회여야 한다.

8.2 DB 구조 비교와 선택

후보	장점	제약	판정
----	----	----	----

후보	장점	제약	판정
Kubernetes 내부 Stateful DB	배포 단위 통합	Cluster Storage/Node 장애와 영속 DB 장애 경계가 결합	제외
MariaDB 직접 연결	구조 단순, Transaction/Constraint 사용	Backend가 DB Node topology와 장애 전환 방식에 직접 결합	대안
외부 MariaDB Primary/Replica + MaxScale	영속 DB 경계 분리, 복제 기반 구조, Application과 DB topology 사이 Routing Endpoint 제공	MaxScale 자체 가용성과 Failover 정책 검증 필요	선택

원 목표 구조는 MariaDB Primary 1대와 Replica 1대, MaxScale 2대로 구성한다. Backend는 개별 DB Node가 아니라 Common VIP 10.1.93.90:3306을 통해 MaxScale 계층에 접근한다. MaxScale은 Application과 DB topology 사이의 Access/Routing 경계를 제공하며 MariaDB Node를 외부 사용자에게 직접 노출하지 않는다.

영속 데이터 접근 흐름

Backend / DATA access

→ Common VIP 10.1.93.90:3306

→ MaxScale-01 / MaxScale-02 논리 계층

→ MariaDB Primary / Replica

→ Transaction·Constraint·Replication

8.3 일관성 처리 원칙

단계	권위 책임	저장/처리 위치
Vote 접수·변경·삭제	VOTE	Redis Current Vote
deadline-집계	VOTE	Redis 상태를 기준으로 selected_position 생성
Move 최종 유효성 확인	GAME	Board·Turn·렌주 규칙 재검증
Move 단일 확정	GAME	Application 먹등성 + MariaDB Transaction/Constraint
Board/Turn 갱신	GAME	DB Commit 성공 후 Redis Runtime State 갱신
분석 작업 생성	GAME	정상 Move 확정 후 game_id + move_no + immutable Board Snapshot(또는 불변 참조)을 Redis Streams에 기록
판세 분석 처리	ANALYSIS	Consumer Group이 Job을 처리하고 결과를 game_id + move_no에 먹등 반영
Realtime 전파	REALTIME	Redis Pub/Sub → Backend Replica → WebSocket

9. Persistent Storage와 Storage 후보 검토

공유 파일형 Storage와 Workload Local Storage, Object Storage를 같은 문제로 취급하지 않는다. 여러 Node에서 재연결할 Shared File PVC/PV는 NFS 계열 또는 분산 Block/File Storage의 문제이고, Local Storage는 소프트웨어의 파일시스템 제약과 단기 보존 요구를 다룬다. MinIO는 Object/Backup 확장 계층의 후보이다.

후보	적용 문제	장점	제약판정
Linux VM 기반 External NFS	Shared File / PVC	구현과 설명이 단순하고 NFS Subdir External Provisioner를 통해 StorageClass/PVC 기반 동적 PV 생성 가능	NFS 자체 SPOF와 단일 Share 의존성 존재 / 현재 선택
NAS/Storage System 기반 NFS	Shared File / PVC	전용 Storage 장비가 제공하는 NFS로 Compute와 Storage 책임 분리 가능	현재 프로젝트에 실제 NAS 구성요소가 없음 / 구조 대안으로 검토
Longhorn 등 분산 Storage	Shared Block/File	Replica 기반 Storage HA	Storage 운영 자체가 별도 프로젝트 수준으로 확대 / 미채택
Local Storage	Node Local / Ephemeral Storage	구조가 단순하고 Prometheus/Loki의 Local Filesystem 제약에 부합	Node 장애·Pod 이동 시 가용성·보존 제약 / 관측성 단기 Storage에만 선택, Local PV·emptyDir 등 구체 유형은 06에서 확정
MinIO	Object / Backup 확장	S3 호환 Object Storage, Artifact/Backup 확장 가능	NFS 대체가 아님 / 향후 확장 후보

Shared File Storage의 현재 선택은 External NFS + NFS Subdir External Provisioner + StorageClass/PVC/PV이다. Provisioner는 기존 NFS Server/Share를 사용해 PVC 요청마다 하위 디렉터리와 PV를 동적으로 생성한다. NFS 소비자는 Jenkins Controller 상태와 Redis AOF 등 Shared File에 적합한 대상으로 한정하며, 모든 Stateful Workload의 기본 저장소로 사용하지 않는다. Provisioner는 NFS HA를 제공하지 않으므로 NFS SPOF와 복구시간은 별도로 검증한다. MariaDB는 독립 DB Storage를 사용하고, Redis AOF의 NFS I/O 영향은 부하·복구 검증에서 확인한다.

구성요소 경계 NAS와 MinIO는 현재 논리 아키텍처의 실제 구성요소로 추가하지 않는다. NAS 기반 NFS는 선택 근거를 위한 구조 대안이고 MinIO는 Object/Backup 확장 후보로만 기록한다.

10. CI/CD·GitOps 및 Image Delivery

역할	선택	책임과 선택 이유
CI	Jenkins - Kubernetes 내부 Workload	Source Build/Test와 Image 생성에 집중. 전용 Jenkins VM 의존성을 제거하고 Worker 장애 시 재스케줄 가능성을 확보
Image Registry	Harbor - 외부 VM	Private Registry와 인증·Image 관리 경계를 Cluster Runtime과 분리
CD	Argo CD - Kubernetes 내부	Deployment Repository의 Git Desired State를 Cluster에 동기화하고 CI의 직접 kubectl 배포를 배제

Jenkins를 Kubernetes 내부 Workload로 선택한 이유는 특정 Jenkins 전용 VM에 대한 의존성과 의도적 SPOF를 줄이고, Worker 장애 시 재스케줄 가능한 실행 경계를 확보하기 위해서다. 다만 Jenkins 자체가 자동으로 완전한 HA가 되는 것은 아니며, 상태 저장과 PVC/NFS 의존성, Worker Failure Domain은 별도의 장애 검증 대상으로 둔다.

Image Flow

Source Repository

→ Jenkins(in-cluster): Build / Test

- Harbor(external): Image Push / Registry
- Kubernetes Runtime: Image Pull

Deployment Configuration Flow

- Deployment Git Repository
- Argo CD(in-cluster): Desired State Sync
- Kubernetes Deployment / Service / Gateway
- ※ Jenkins는 Cluster Desired State를 직접 적용하지 않음

11. Infrastructure Automation

Ansible은 사용자 정상 요청 경로와 GitOps CD에서 분리된 Infrastructure Automation 역할이다. 운영자가 반복해야 하는 Host, Kubernetes, External Service 구성을 코드화하여 수동 대비 구축시간·재구축시간·직접 개입·실패 Task를 비교할 수 있게 한다.

자동화 영역	이 문서의 책임 범위	자동화·테스트 설계에서 구체화
Host Configuration	OS 기본설정·패키지·계정·시간 동기화·필수 서비스	Inventory, Role, 변수, Idempotency
Kubernetes Bootstrap	kubeadm 사전 구성, Control Plane/Worker 초기화	실행 순서·검증·재실행 정책
Cluster Add-on Bootstrap	Calico, NFS Subdir External Provisioner, Jenkins, Argo CD 등 기반 구성요소 초기 적용	공식 Manifest/Helm 자산 활용 범위와 검증 순서
External Infrastructure	MariaDB/MaxScale, NFS, Harbor	Role·Template·Secret 처리·Health Check
Recovery Assist	장애 서버 환경 재구축과 Restore 연결	재구축 시간·RTO 측정 절차

Application Desired State의 지속 적용은 Argo CD에 두고, Ansible은 Cluster와 외부 서비스가 그 Desired State를 수용할 수 있는 기반을 반복 가능하게 준비하는 데 집중한다.

12. Observability

영역	선택	책임
Metrics	Prometheus	Infrastructure/Kubernetes/Application Metric 수집·저장·질의
Visualization	Grafana	핵심 KPI와 진단 Metric Dashboard
Logs	Grafana Alloy + Loki	Application·Kubernetes·주요 External Infrastructure의 운영 로그를 수집·중앙 조회한다. Ansible 실행 결과와 실험 Raw Evidence는 자동화·검증 기록으로 별도 보존하고, 필요한 장애 시각과 식별자를 Observability 데이터와 연계한다.
Kubernetes Object	kube-state-metrics	Deployment/Pod/Node 등 Object 상태 Metric
Resource Metrics API	Metrics Server	Kubernetes Resource Metric 제공
Host	node_exporter	CPU/Memory/Disk/Network 진단 Metric
Alerting	Prometheus Alert Rules + Alertmanager	주요 장애·가용성 저하·자동 복구 실패 상태를 평가하고, 운영자 대응이 필요한 Alert를 그룹화·억제·라우팅

Prometheus 공식 Storage 제약에 따라 TSDB는 NFS에 두지 않고 Worker Local Filesystem에서 단기 보존한다. Loki는 소규모 PoC에 적합한 Single Binary + Local Filesystem을 사용하고 Shared NFS를 HA Storage로 사용하지 않는다. Grafana Dashboard·Datasource는 GitOps Provisioning으로 복원하며 기본 PVC를 두지 않고, Alertmanager도 1차 범위에서 기본 PVC를 두지 않는다. Thanos·Mimir 등 장기·HA 관측 저장소는 4인·4주 범위 대비 운영 복잡도가 크므로 미도입하고, 핵심 실험 결과는 CSV·JSON·시간표식·실행 로그 등 Raw Evidence로 별도 보존한다.

ELK와 Full Distributed Tracing Stack은 기본 범위에서 제외한다. 현재 핵심 검증은 Vote 처리량·지연·오류, Backend 장애복구, Resource 사용량, AI 분석 처리시간·실패율, stale 분석 오반영 여부와 구조화 사건 로그로 우선 증명할 수 있다. Tracing은 실제 병목 분석에서 필요성이 확인될 때 확장 후보로 남긴다.

검증 주장	주요 Metrics/Logs
Vote 성능	Vote events/s, p95/p99, Error rate, Active Room/Game, 동시 실시간 연결
Backend 장애	장애 시각, 실패 요청, 재연결 성공률, 상태 복구시간, 잘못된 사용자 패배 0건, Alert firing/resolved 시각 또는 장애 감지시간
AI 판세 분석 격리	analysis latency, failure rate, Stream pending/consumer lag, stale 현재 상태 오반영 0건, 분석 실행 중 게임 핵심 경로 지연 변화
Ansible	Playbook OK/Changed/Failed, 구축·재구축 시간, 직접 개입 단계
DR	Backup/Restore 시각, 데이터 무결성, RTO/RPO

12.1 장애 감지와 운영 알림 원칙

모든 장애 사건을 운영자 알림 대상으로 취급하지 않는다. 자동 복구되어 서비스 영향이 해소되는 일시 장애는 Metrics·Logs·Event에 기록하여 추적 가능하게 하고, 반복 발생이나 가용성 저하가 확인되면 Warning 대상으로 평가한다. 자동 복구 실패, 서비스 가용성 상실, 데이터 손실 위험, 단일 장애점 장애 등 운영자 개입이 필요한 상태는 Critical Alert 대상으로 구분한다. Prometheus Alert Rules가 조건을 평가하고 Alertmanager가 발생한 Alert의 그룹화·중복 억제·라우팅을 담당한다. 세부 임계값, 지속시간, Severity 및 실제 통보 채널은 자동화·테스트 설계에서 확정한다.

13. Configuration·Security

영역	적용 원칙	결정 이유
외부 서비스 통신	HTTPS/WebSocket, TLS는 Gateway Layer에서 종료	Client 진입점을 단일화하고 내부 Service와 외부 노출을 분리
일반 설정	ConfigMap 등 외부 설정	Image와 환경별 설정 분리
서비스·인프라 Secret	Kubernetes Secret 등 실행환경의 Secret 관리 구조 사용	DB·Registry 등 Runtime Credential과 인증서·비밀값을 일반 설정에서 분리한다. Member 비밀번호는 DATA에서 password hash로 별도 관리하며 평문으로 저장하지 않는다. 고도화 Secret Platform은 기본 범위에서 제외한다.
Pod 간 통신	Calico NetworkPolicy	Namespace만으로 보안 경계를 가정하지 않음
운영 권한	Kubernetes RBAC	Operator·Service Account 최소 권한
DB 접근	Backend → Common VIP:3306 → MaxScale 경로만 허용	Application과 MariaDB Node topology·외부 사용자 접근 분리
Registry	Harbor HTTPS + 인증	Image Supply Chain 접근 경계 유지

보안 설계 경계 이 문서는 보안 책임과 통신 경계를 확정한다. 실제 Source/Destination, Host Firewall, 세부 Port Matrix, Secret 배포 자동화는 물리 네트워크 및 자동화 설계에서 구체화한다.

14. DR·Backup·Recovery

Replication/Failover는 Availability를 높이는 구조이고 Backup은 데이터 손상·삭제까지 복구하기 위한 별도 축이다. MariaDB Replica와 MaxScale 2대가 존재하더라도 Backup/Restore 검증은 유지한다. Runtime State, Persistent Data, Cluster State는 수명과 복구 목표가 달라 각각의 방식으로 다룬다.

복구 대상	선택 방식	복구 목표
Persistent Data: MariaDB	mariadb-backup → 보호된 Backup Storage → Restore	Member·MemberStats·Move·GameResult·RatingHistory 무결성 복원
Runtime State: Redis	AOF Persistence → Runtime Restore	활성 Room/Game/Turn/Board/분석 최신 상태 복구 가능성 확인
Cluster State: etcd	etcd Snapshot → Control Plane State Restore	Kubernetes Object/Cluster State 복원
Shared File: NFS	파일/Volume Backup 정책은 물리 Storage 설계와 연계	Jenkins 등 PVC 의존 데이터의 손실 범위와 복구 기준 확정

Backup 위치·주기·Retention과 RTO/RPO 목표 수치는 실제 저장 용량과 물리 Failure Domain, 복구 실험을 바탕으로 후속 단계에서 확정한다.

15. 장애 영향·동시성·AI 격리 검증

15.1 주요 Failure Point

Failure Point	직접 영향	논리적 대응	검증
Common VIP / Keepalived / HAProxy	Service/API/DB Endpoint 신규 진입 영향	Keepalived VRRP로 VIP 소유권 전환 + HAProxy Backend Health 기반 포트별 전달	VIP 전환시간, 기존/신규 연결 영향, Backend Health 반영시간
Gateway Data Plane	HTTP/WebSocket 신규 진입 실패	Gateway Replica와 Kubernetes Service	서비스 진입 복구시간
Backend Pod	API/WebSocket 연결 손실	다른 Replica + Redis State Reload + 재연결	실패 요청 수·재연결 성공률·상태 복구시간·잘못된 사용자 패배 판정 0건
ANALYSIS Workload	판세 표시 지연·실패	게임 권위 경로와 분리, 실패 상태 기록, 재시도/다음 Move 수렴	Turn·Move·Result 무영향, stale 오반영 0건
Redis	Guest/Member 인증 세션, Room/Game/Turn/Vote/분석 최신 상태, Pub/Sub, Analysis Stream 영향	AOF 기반 복구 후 Current State 및 Stream Pending/Consumer 복구 절차 검증	MTTR·상태 복원·Analysis Job 중복/유실 영향
MaxScale 2대 중 1대 장애	일부 DB Routing Endpoint 손실	HAProxy Health Check로 장애 MaxScale을 Backend Pool에서 제외하고 정상 MaxScale로 전달	접속·Transaction 영향과 전환시간
MariaDB Primary	Read/Write/Transaction 영향	Replica 기반 승격/복구 + 필요 시	Failover·RTO·데이터 무결성

Failure Point	직접 영향	논리적 대응	검증
		Backup Restore	
NFS	PVC 기반 Stateful Storage 접근 실패	NFS 복구·Mount 재연결·재구축	SPOF·MTTR·PVC 영향
Worker Node	해당 Node Pod 중단	다른 Worker로 재스케줄링	Backend/ANALYSIS/Jenkins 재배포 영향
Jenkins Workload/PVC	신규 Build/Test·Image 생성 중단	Pod 재스케줄·PVC 재연결. Runtime Game은 직접 영향 없음	CI 복구시간과 NFS/PVC 의존성

15.2 Vote → Move → Analysis 일관성

권위 처리 순서

Turn VOTING

→ VOTE: participant_id + game_id + turn_no 기준 마지막 유효표 관리

→ server deadline: 집계 고정

→ VOTE: selected_position 생성 또는 0표 Pass

→ GAME: Board/Turn/렌주 유효성 재검증

→ 정상: MariaDB Move 1개 Commit / Pass: Move 0개

→ Redis Board/Turn Runtime State 갱신

→ 정상 Move만 Redis Streams에 ANALYSIS Job 기록(game_id + move_no) → analysis_status=PENDING

→ ANALYSIS Consumer Group 처리

→ 성공 시 READY + 승률 반영 후 ACK / 재시도 가능 실패 시 PENDING 유지·Retry / 최종 실패 시 FAILED

→ READY 결과 또는 FAILED 상태를 game_id + move_no 기준으로 Current State에 먹등 반영

→ Redis Pub/Sub → REALTIME 최종 상태 전파

검증 지점	보장 방식	성공 기준
Vote 변경/삭제	participant_id + game_id + turn_no 현재 표 관리	사용자별 마지막 유효 표 1개
deadline 경합	서버 권위 deadline 이후 요청 거부	deadline 이후 도착한 신규·변경 투표의 집계 반영 0건
금수·점유 좌표	GAME의 유효 좌표 기준을 VOTE가 접수 단계에서 적용, Commit 전 GAME 재검증	유효하지 않은 Move 0건
단일 Move	먹등 키 + MariaDB Transaction/Constraint	정상 Turn Move 1건, Pass 0건
Result/Rating	game_id 결과 원장 + 중복 처리 방지	GameResult 1회, RatingHistory 중복 0건
Realtime 수렴	DB Commit 후 Redis 상태 갱신·Pub/Sub, 재접속 Current State 재조회	다른 Replica/재접속 사용자가 동일 최종 상태 확인
AI stale 격리	Redis Streams Consumer Group + game_id + move_no 먹등 처리, 현재 표시 대상 비교	재처리 중복 오염 0건, 과거 분석이 현재 Board 결과로 오반영 0건

MariaDB Move Commit 이후 Redis Runtime State 갱신 또는 Analysis Job 기록 전에 처리 인스턴스가 중단될 수 있으므로, 동일 game_id + turn_no 처리 재시도 시 기존 확정 Move를 조회하여 새 Move를 생성하지 않고 Redis Board/Turn 상태와 필요한 Analysis Job을 해당 move_no 기준으로 재동기화한다. Analysis Job의 중복 전달은 game_id + move_no 먹등 처리로 수렴한다.

15.3 장애 감지·알림 정책

상태	관측·기록	Alert 수준	기본 대응
Pod 단발성 Restart 후 정상 복구	Metrics/Logs/Event	없음	자동 복구 결과 추적
Pod Restart 반복 / Replica 감소 지속	Metrics/Logs	Warning	원인 확인

상태	관측·기록	Alert 수준	기본 대응
LB 장애 후 VIP Failover 성공, 서비스 정상	Metrics/Event	기록 또는 Warning	Failover 결과 확인
VIP Failover 실패 / Service Endpoint 불가	Metrics/Logs	Critical	운영자 개입
ANALYSIS 단발 실패	Metrics/Logs	없음	다음 분석 및 실패를 추적
ANALYSIS 실패율 지속 증가	Metrics	Warning	Resource/Consumer 상태 확인
MaxScale 한 대 장애, 정상 Backend로 우회	Metrics/Logs	Warning	이중화 열화 확인
MariaDB Write 불가/Failover 실패	Metrics/Logs	Critical	DB 복구
NFS 접근 불가	Metrics/Logs	Critical	Storage 복구
Worker 한 대 장애 후 Workload 정상 재배치	Metrics/Event	Warning	Capacity/재배치 상태 확인
Backup 실패	Logs/Metrics	Warning, 반복 시 Critical	Backup 원인 및 복구 확인

16. 전체 논리 아키텍처

A. 사용자 서비스 경로

USER

→ DNS

→ Common VIP 10.1.93.90:443

→ Keepalived VRRP가 활성 LB의 VIP 소유권 유지

→ HAProxy L4

→ Kubernetes Gateway API / NGINX Gateway Fabric

└→ Frontend Nginx: WEB/UI 제공

└→ FastAPI Backend: API/WebSocket 및 도메인 처리

└→ Redis STATE

└→ MariaDB DATA

└→ 정상 Move 확정 시 Redis Streams → ANALYSIS

→ REALTIME → USER

※ 10.1.93.90:80은 HTTPS Redirect 진입점으로 사용

B. Kubernetes 제어 경로

OPERATOR / AUTOMATION

→ Common VIP 10.1.93.90:6443

→ Keepalived VRRP → HAProxy L4

→ Kubernetes API

→ Control Plane

→ Deployment / Service / Gateway / NetworkPolicy / RBAC

C. 영속 데이터 경로

FastAPI Backend

→ Common VIP 10.1.93.90:3306

→ Keepalived VRRP → HAProxy L4

→ MaxScale 2대 논리 계층
 → MariaDB Primary / Replica
 → Transaction / Constraint / Replication
 → mariadb-backup / Restore

D. 게임 상태·분석 경로

VOTE → GAME
 → Move Commit / Board Snapshot
 ↳ Redis Current State → Pub/Sub → REALTIME
 ↳ Redis Streams Analysis Job → ANALYSIS Consumer Group → ACK/Pending·Retry
 → BoardAnalysis 최신 결과 → Redis Current State → Pub/Sub → REALTIME

E. CI/CD 경로

Source Repository → Jenkins(in-cluster) → Harbor(external)
 Deployment Repository → Argo CD(in-cluster) → Kubernetes
 Ansible → Host / Kubernetes Bootstrap / MariaDB·MaxScale / NFS / Harbor

F. 관측·복구 경로

Prometheus / Exporters → Metrics → Grafana
 ↳ Alert Rules → Alertmanager → Operator Notification
 Alloy → Loki → Grafana
 Prometheus TSDB·Loki Log Store → Worker Local Storage(단기 보존)
 실험 Raw Evidence → 별도 검증 기록으로 보존
 MariaDB Backup + Redis ACF + etcd Snapshot → 독립 복구 실험

16.1 장애 경계 요약

- ANALYSIS 장애는 판세 표시만 영향받고 Vote·Move·Turn·Result·Rating의 권위 경로에는 직접 영향이 없어야 한다.
- REALTIME 연결 인스턴스 장애는 Current State 원본 손실로 이어져서는 안 된다.
- MariaDB 복제/MaxScale은 Availability 축이고 Backup/Restore는 데이터 손상·삭제 복구 축으로 분리한다.
- Jenkins in-cluster는 전용 VM SPOF를 제거하지만 Worker와 NFS/PVC Failure Domain에 편입되므로 재스케줄·Storage 복구를 별도로 검증한다.
- External NFS는 현재 Shared File Storage 선택이지만 SPOF를 숨기지 않고 복구시간을 측정한다.
- Alertmanager 장애는 사용자 서비스의 게임 권위 경로에 영향을 주지 않지만 운영자 능동 통보 기능을 저하시킨다. Metrics·Logs 원본 관측 경로와 Alert 전달 경로를 구분하여 장애 영향을 판단한다.

17. 최종 선택 요약 및 후속 설계 이관

17.1 기술 선택 요약

영역	최종 선택	주요 대안·제외	결정 이유
Runtime Platform	Kubernetes / kubeadm	VM 직접 배포	Replica·Service·Health·표준 Network/Storage와 장애 실험 구조
Gateway	Gateway API + NGINX Gateway Fabric	별도 L7 Proxy, Ingress	HTTP/WebSocket L7 Routing 책임 일원화
External Endpoint	Common VIP 10.1.93.90 + Keepalived VRRP + HAProxy Port별 L4	3개 별도 VIP, HAProxy 단독, Keepalived/IPVS 중심	단일 외부 주소에서 VIP Failover와 Port별 L4 전달·Health Check 책임을 분리
CNI	Calico	Flannel, Cilium	NetworkPolicy 요구와 복잡도 균형

영역	최종 선택	주요 대안·제외	결정 이유
Application	FastAPI Modular Monolith	역할별 Microservice	논리 책임 경계를 유지하며 배포 복잡도 통제
Analysis	독립 in-cluster ANALYSIS Workload + Redis Streams Consumer Group	동기 모듈, Backend 내부 Task, Redis Pub/Sub, 별도 Message Broker	게임 권위 경로와 실패·자원 격리, Job 보존·ACK·재처리를 기존 Redis로 구현
Runtime State	Redis + AOF + Pub/Sub (State/Realtime) + Streams (Analysis Job)	Local Memory, DB 중심 상태	다중 Pod 상태 공유·재접속 복구·Realtime 알림과 Analysis Job 전달을 목적별 자료구조로 분리
Persistent Data	MariaDB Primary/Replica + MaxScale 2	MariaDB 직접 연결, K8s 내부 DB	영속 DB 경계와 DB Access/Routing 복제 기반 구조
Persistent Storage	External NFS + NFS Subdir External Provisioner / Prometheus-Loki Worker Local Storage	Longhorn, NAS 기반 NFS, 장기·HA 관측 저장소	Shared PVC는 NFS로 단순화하되 TSDB·Log Store는 Local Filesystem 제약과 단기 보존 범위를 적용. Storage 과설계를 통제하고 실험 Evidence는 별도 보존
Object Storage	현재 미도입 / MinIO 확장 후보	NFS 대체로 사용	Object/Backup 용도는 Shared File Storage와 별도 문제
CI/CD	Jenkins(in-cluster) + Harbor(external) + Argo CD(in-cluster)	CI 직접 배포	Build/Image/Git Desired State 책임 분리와 Jenkins 전용 VM 의존 제거
Observability	Prometheus + Alertmanager + Grafana + Loki/Alloy	ELK, Full Tracing, 장기·HA 관측 저장소	핵심 KPI·로그 증거를 단기 Local Storage에 수집하고, 운영자 개입이 필요한 이상 상태를 능동적으로 통보. 핵심 실험 Evidence는 별도 보존
Automation	Ansible	수동 구축	On-premise 반복 구축·재구축 Before/After 측정

17.2 물리 아키텍처로 이관

항목	구체화할 내용
Kubernetes	Control Plane/Worker 수와 Physical Server/VM 배치, Worker 1대 장애 시 수용 가능성
Common VIP/L4	LB-01/LB-02의 Physical Server 배치와 Failure Domain, VRRP 인터페이스·우선순위, HAProxy Backend 대상/Port, External IP·NIC·Firewall/Port Matrix
MariaDB/MaxScale	Primary/Replica와 MaxScale 2대의 Physical Failure Domain, DB Storage, 장애 전환
Storage	NFS VM 용량·배치·SPOF와 Jenkins/Redis PVC 영향, Prometheus-Loki Worker Local Storage·Retention, Grafana GitOps 복원 경계
Network	Node/VM Subnet, Pod/Service CIDR, DNS, External L2 주소 및 Firewall, LB 간 L2 연결·VRRP 통신·VIP 이동·Gratuitous ARP 허용 여부와 미지원 시 단일 LB 고정 IP 적용 경계
Resource	Frontend/Backend/Redis/ANALYSIS/Jenkins/Observability의 CPU·Memory·Disk와 Worker Capacity

항목	구체화할 내용
Management	Harbor-NFS-Ansible Controller 등 외부 VM의 실제 배치와 장애 영향

17.3 자동화·테스트 설계로 이관

항목	구체화할 내용
Ansible	Inventory, Role, Playbook, 변수·Secret, Idempotency, 재구축 절차
Kubernetes	kubeadm 상세 설치, Calico/NFS Subdir External Provisioner/Jenkins/Argo CD 등 Bootstrap·Health 검증
Application	Deployment/Service/Gateway Manifest, Resource Request/Limit, HPA 조건
Analysis	선택한 판세 분석 방식의 구체 알고리즘·파라미터 검증, Redis Streams Consumer Group/Pending/ACK·Retry, stale·중복 격리 테스트, Resource 부하
Database	MariaDB Replication/Failover, MaxScale Routing/Health, Backup/Restore
Observability	FastAPI·Analysis Metric 이름, Dashboard, 장애·부하 실험용 Log 식별자, Prometheus·Loki Local Storage·Retention·Evidence Export, 외부 VM Alloy Agent 배치, Alert Rule·Severity·지속시간·Grouping·Routing, 실제 Notification Channel, Alert firing/resolved 검증
실험	Vote 부하, Backend/Node/Analysis/DB/MaxScale/NFS/Jenkins 장애, DR, Ansible Before/After

설계 결론 서비스 규칙, 핵심 문제·검증축, 논리 역할에 대해 구현 기술과 논리 책임이 연결되어 있으며, Common VIP·DB·CI/CD·AI 분석·Storage의 원 목표 구조가 서로 모순되지 않는다. 후속 물리 아키텍처에서는 이 논리 구조를 기준으로 배치·자원·네트워크·Failure Domain을 구체화한다.